Programado el frontend con Angular

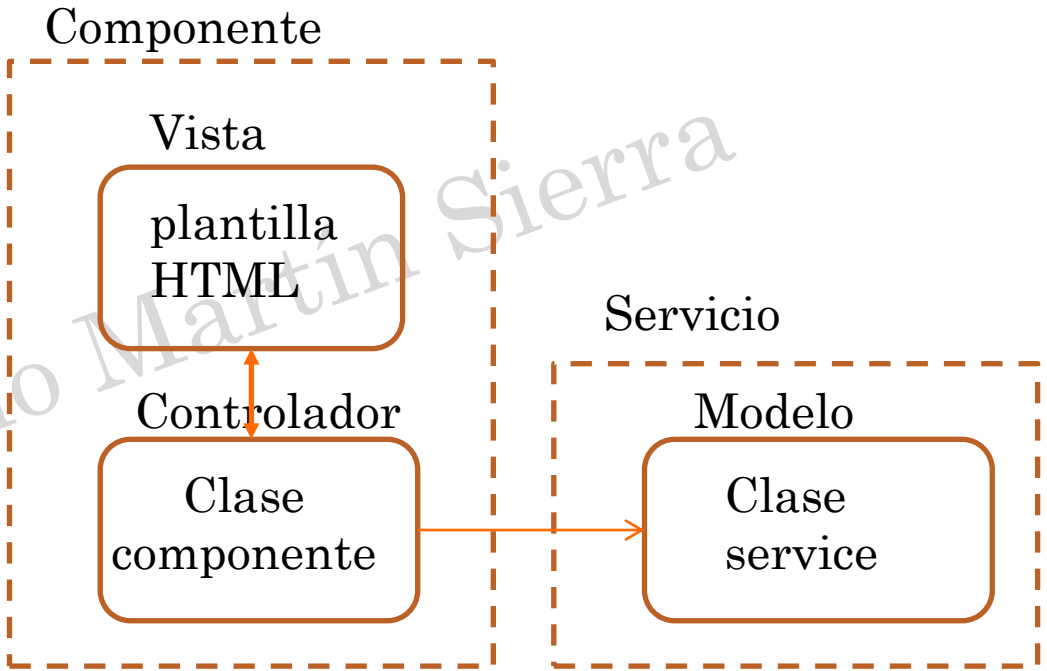
Características de Angular

- Framework para la creación de aplicaciones Web de lado cliente
- Basado en la creación de plantillas HTML gestionadas por componentes
- Simplifica la interacción con el usuario
- Vinculación a datos
- Implementación de código mediante TypeScript (superconjunto JavaScript orientado a objetos)
- Aplicaciones basadas en patrón MVC

Antonio Martín Sierra

Patrón MVC

- **Modelo: lógica de negocio**
- **Controlador: gestión de eventos de usuario**
- **Vista: generación de respuestas**



Antonio Martín Sierra

Configuración

- **Instalación de Node.js. Servidor para publicación de aplicaciones Angular. Incluye npm, una herramienta para gestión de dependencias. Descargable en:**

  https://nodejs.org/es/download/

- **Angular cli. Herramienta para crear proyectos angular. Se instala con npm:**

  > npm install -g @angular/cli

- **Visual Studio Code. Editor de proyectos angular para facilitar la codificación.**

Antonio Martín Sierra

VERSIONES

- **Se puede comprobar las versiones de herramientas mediante el comando:**

>ng version



- **Para actualizar últimas versiones:**

>npm install -g npm@latest
>npm install -g @angular/cli

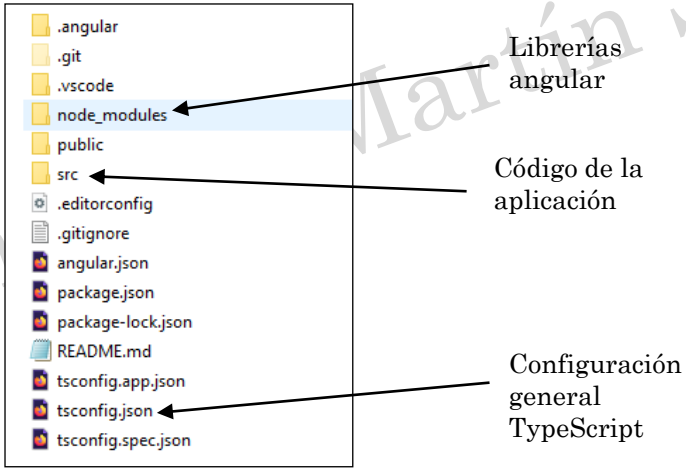
Para actualizar la versión de angular, se vuelve a instalar el cli

Creación de una aplicación

- **Se crea desde línea de comandos utilizando:**

  >ng new nombreApp

- **Se crea una estructura similar a esta:**



Antonio Martín Sierra

Creación de una aplicación con otra versión

- **Angular solo permite tener una versión global instalada**
- **Pero se puede crear un proyecto que utilice una determinada versión de angular, sin instalar dicha versión. Solo se usará en ese proyecto puntual**
- **El siguiente ejemplo crea un proyecto con Angular 20, pero sin instalar la versión:**

```
>npx @angular/cli@20 new proyecto20
```

- **Si queremos actualizar un proyecto existente a una determinada versión angular, en la carpeta de proyecto:**

```
> ng update @angular/core@22 @angular/cli@22
```

Antonio Martín Sierra

Workspace

- **Las aplicaciones se pueden crear dentro de un workspace para compartir librerías**
- **Un workspace se crea:**

```
>ng new nombre-workspace --create-application=false
```

- **o:**

```
>ng new nombre-workspace --no-create-application
```

- **Para crear una aplicación, dentro de la carpeta del workspace:**

```
>ng generate application nombre-app
```

- **Y para lanzarla:**

```
>ng serve nombre-app -o
```

Antonio Martín Sierra

VARIABLES Y TIPOS TYPESCRIPT

- **Se declaran:**

```
let variable:tipo;
```

- **Los tipos básicos de TypeScript son: boolean, string y number.**
- **Se puede declarar una variable asignándole un valor, o hacerlo más adelante**

```
let cad:string;
cad="hello";
let n:number=25;
```

- **Si queremos una constante, se utiliza const en lugar de let:**

```
const PI=3.14.16;
```

Antonio Martín Sierra

Instrucciones de control TypeScript

if

```
if(a>3){
  console.log("si");
}else{
 console.log("no");
}
```

for

```
for(let a=1;a<=10;a++){
  console.log(a);
}
```

for of

```
for(let m of lista){
  console.log(m);
}
```

switch

```
switch(a){
  case 1:
    console.log("uno");
    break;
  case 2:
    console.log("dos");
    break;
  default:
    console.log("otro");
}
```

while

```
while(r<10){
  console.log(r);
  r++;
}
```

do while

```
do{
  a=input("número:")
}while(n<0);
```

Listas y mapas TypeScript

- **Lista: Secuencia de elementos de un mismo tipo. No tiene tamaño fijo:**

```
const lista:number[]=[3,7,2,4];
lista.push(10);
console.log(lista[4]);//10
for(n of lista){
    console.log(n);
}
```



Principales métodos de una lista: length, push, pop, splice, map, filter, …

- **Mapa: Cada elemento tiene asociada una clave**

```
const map = new Map<string, number>();
map.set('one', 1);
map.set('two', 2);
console.log(map.get('one')); // 1
console.log(map.has('two')); // true
```

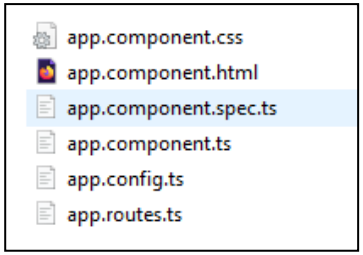


Principales métodos de un map: size,set,delete,get, keys, values, …

Antonio Martín Sierra

COMPONENTES

- Al crear un proyecto se genera automáticamente un componente dentro de la carpeta src:



- Un componente define el aspecto de una página de la aplicación y controla su comportamiento

- Consta de dos elementos:
  - Plantilla. Archivo HTML que genera la vista de la página
  - Componente. Clase TypeScript donde se implementa la funcionalidad de la página

Antonio Martín Sierra

Plantilla

- **Bloque HTML que forma un componente y establece el aspecto de la página.**
- **Puede incluir vínculos hacia la clase del componente para generar dinámicamente contenido y suministrar datos de usuario a dicha clase**

Martín Sierra

```
<div class="formulario">
  <div>
      <label for="">Introduce código: </label>
      <input type="text" [(ngModel)]="codigo">

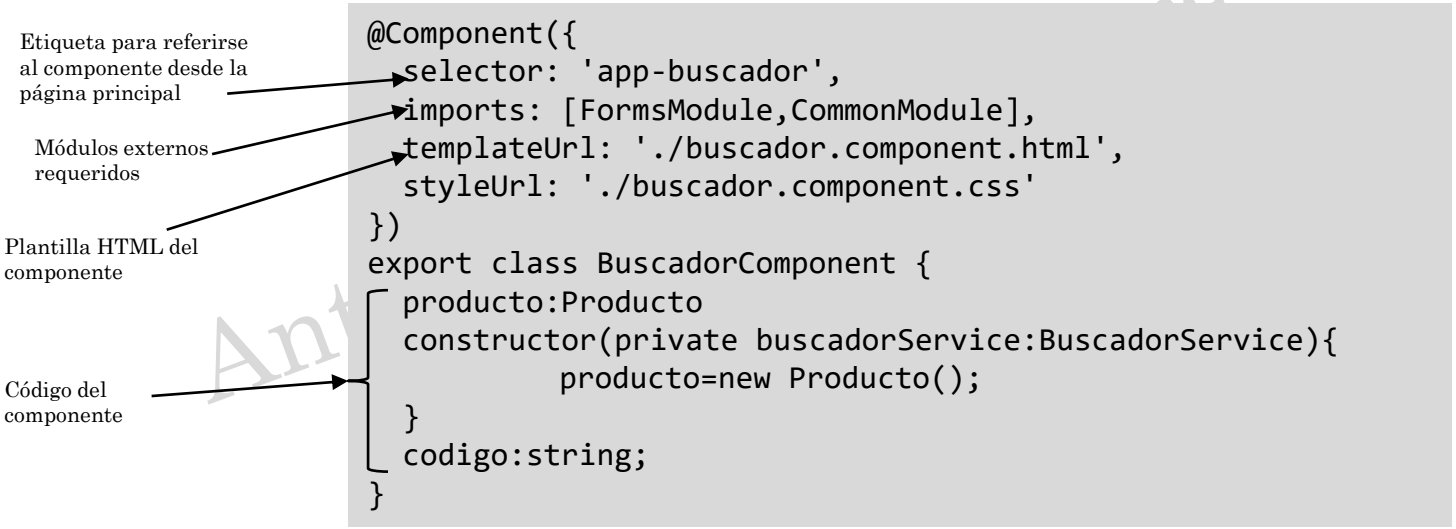
      <br>
      <p><a>{{producto.name}}</a><br>
         <a>{{producto.precio}}</a>
      </p>
  </div>
</div>
```

Hacia la clase componente

Volcado desde el componente hacia la página

Clase del componente (controlador)

- **Define el comportamiento de la página, recogiendo datos de esta, respondiendo a evento y generando resultados**



La página index.html

- Es la página que se carga en el navegador al ejecutar una aplicación Angular
- Contiene referencias a componentes mediante la etiqueta de selector

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Aplicación Angular</title>
  <base href="/">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <app-buscador></app-buscador>
</body>
</html>
```

Referecia al componente que será procesado al solicitar la página

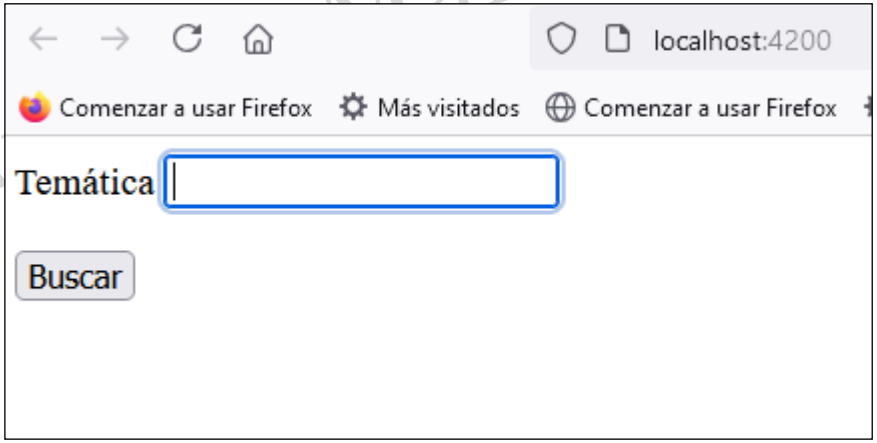
Antonio Martín Sierra

Ejecución de la aplicación

- **Para ejecutar una aplicación angular utilizaremos el comando:**

```
>ng serve -o
```

- **La aplicación se ejecutará automáticamente en un servidor node.js, se abrirá un navegador y se lanzará una solicitud de index.html**



A. Martín Sierra

VINCULACIÓN A DATOS

- **A través de directivas asociamos el contenido de controles HTML de la platilla a propiedades del componente**

```
<input type="text" [(ngModel)]="nombre">
```

Se debe importar el módulo *FormsModule* en el componente

- **Mediante los interpoladores se vuelcan propiedades del componente en la página**

```
<h2>{{texto}}</h2>
```

```
export class DatosComponent {
  nombre:string;
  texto:string;
  :
}
```

Antonio Martín Sierra

Paso de parámetros al componente

- **Se pueden pasar parámetros al componente desde la página index mediante atributos de la etiqueta asociada, utilizando *property binding***

```
<app-root [level]="5"></app-root>
```

```
export class DatosComponent {
  level:number;
  :
}
```

Antonio Martín Sierra

EVENTOS

- **Una de las funcionalidades de la capa front es capturar eventos o acciones de usuario.**
- **En angular los eventos se manejan a través de funciones de respuesta definidas en el componente**

Antonio Martín Sierra

```
<input type="button" value="Buscar" (click)="buscar()">
```

```
export class BuscadorComponent {
  buscar():void{
      :
   }
}
```

Directiva ngIf

- Puede ser incluida en cualquier etiqueta HTML para que dicha etiqueta sea o no procesada en función de una condición.
- Su formato:

```
<etiqueta *ngIf="expresion">...</etiqueta>
```

- Si expresion es evaluada como falso, la etiqueta será eliminada del árbol de objeto DOM.
- Para poder utilizar esta etiqueta es necesario importar el CommonModule en el componente:

```
@Component({
  selector: 'app-buscador',
  imports: [CommonModule],
  :
```

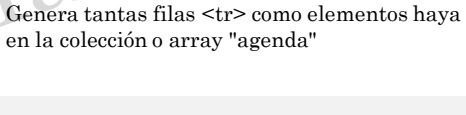
Antonio Martín Sierra

Directiva ngFor

- Se incluye en una etiqueta para que esta aparecerá tantas veces como se indique en la expresión de iteración asignada a la directiva.
- Su formato:

```
<etiqueta *ngFor="let variable of array">...</etiqueta>
```

- Ejemplo:




```
<tr *ngFor="let c of agenda">
      <td>{{c.nombre}}</td><td>{{c.edad}}</td><td>{{c.telefono}}</td>
</tr>
```

- El uso de esta directiva también requiere la importación de CommonModule

Antonio Martín Sierra

Directiva ngSwitch

- Utilizamos esta directiva para mostrar diferentes elementos de un conjunto de ellos, en función del resultado de una expresión.
- Procesa la etiqueta cuyo valor @ngSwitchWhen coincide con el resultado de la expresión:

Martín Sierra

```
<div [ngSwitch]="seleccionado.nombre">
  <div *ngSwitchWhen="Ana">Vive cerca</ div >
  <div *ngSwitchWhen="Belén">Acaba de mudarse</ div >
  <div *ngSwitchWhen="Marcos">Vive ahí desde siempre</ div >
  <div *ngSwitchDefault>desconocidos</ div >
</div>
```

Control flow

- **Desde Angular 19, se incluye las sentencias control flow para controlar el flujo de ejecución en una vista**
- **En Angular 21 se recomienda utilizar esta técnica en lugar de directivas ngIf, ngFor y ngSwitch**

```
<etiqueta *ngIf="expresion">
...
</etiqueta>
```



```
@if(expresion){
   <etiqueta>…</etiqueta>
}@else{..}
```

Antonio Martín Sierra

```
<etiqueta *ngFor="let variable of array">
...
</etiqueta>
```



```
@for(n of array; track n){
   <etiqueta>…</etiqueta>
}
```

La librería Zone.js

- Librería de angular que se encarga de detectar cambios en la aplicación para notificarlos a la interfaz gráfica
- Esta librería ya NO está presente en Angular 21
- Durante la gestión de eventos, Angular notifica automáticamente los cambios a la interfaz
- Si se producen cambios externamente (temporizadores, suscripción a observables,..) se deberán utilizar signals para notificar cambios

Antonio Martín Sierra

SIGNALS

- **Un objeto reactivo que almacena un valor**
- **Cuando cambia el valor, el signal notifica el cambio (señal) a quien lo está utilizando**
- **Creación. Se crea con la función signal:**

```
dato=signal(10);
name=signal("test");
```

- **A través de los métodos del signal (set, update) podemos modificar el valor del signal**

```
dato.update(n=>n+1);
name.set("nuevo valor");
```

- **Desde la página se accede a el como a una función:**

```
{{dato()}}
```

Antonio Martín Sierra

Signals. Funciones derivadas

- **Las funciones derivadas permiten asociar valores a cambios en un signal**
- **Dos funciones principales que se ejecutan al cambiar un signal:**
  - **computed(). Devuelve un valor calculado a partir de un signal.**

```
mensaje=computed(()=>dato()+" nuevo valor");
```

  - **effect(). Provoca un efecto secundario al cambiar un signal. No devuelve valor**

```
effect(() => console.log('Nuevo nombre:', name());
```

Antonio Martín Sierra

VINCULACIÓN A DATOS CON SIGNALS

- **Usando signals se cambia la forma de recoger y asignar datos desde la vista al componente:**

```
<input type="text" [ngModel]="nombre()" (ngModelChange)="nombre.set($event)">
```

- **El interpolador llama al signal como si fuera una función:**

```
<h2>{{texto()}}</h2>
```

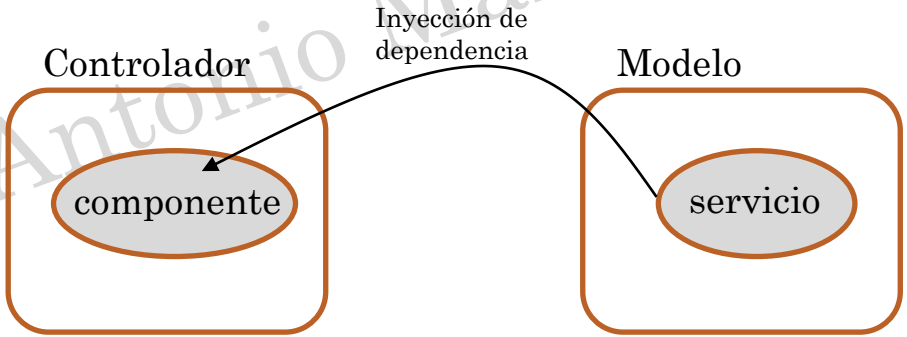
```
export class DatosComponent {
  nombre=signal<string>("");
  texto=signal<string>;
  :
}
```

Antonio Martín Sierra

SERVICIOS

- **Encapsulan la lógica de negocio de la aplicación (Modelo).**
- **Exponen su funcionalidad al componente controlador a través de métodos.**
- **El servicio es inyectado en el componente para que pueda hacer uso del mismo**

Antonio Martín Sierra



Implementación de un servicio

- **Se implementa en una clase estándar anotada con @Injectable:**

```
@Injectable({
  providedIn: 'root'
})
export class BuscadorService {
  buscar(tematica:string):String []{
    :
  }
}
```

- **En el componente se inyecta a través del constructor:**

Inyección de servicio en un atributo de la clase

```
export class BuscadorComponent {
  constructor(private buscadorService:BuscadorService){

  }
```

Servicios en Angular 22

- **Cambia la definición de la clase:**

```
@Service()
export class BuscadorService {
  buscar(tematica:string):String []{
    :
  }
}
```

- **En el componente se puede seguir inyectado a través del constructor:**

```
export class BuscadorComponent {
  constructor(private buscadorService:BuscadorService){

  }
```

Peticiones Http en Angular 22

- **En un servicio creado con @Service() no está permitida la inyección por constructor, se debe utilizar la función *inject()*.**
- **La inyección no se hace en constructor, sino en un inicializador de propiedad:**

```
import { HttpClient } from '@angular/common/http';

@Service()
export class BuscadorService {
  http:HttpClient=inject(HttpClient);
}
```

- **Igualmente, hay que declarar el proveedor:**

```
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [provideHttpClient(),…]
};
```

Peticiones Http

- Se realizan a través del componente HttpClient incluido en la librería http.
- Este componente deberá ser utilizado desde un servicio. Puede ser inyectado a través del constructor:

```
import { HttpClient } from '@angular/common/http';

@Injectable({
  providedIn: 'root'
})
export class BuscadorService {
  constructor(private http:HttpClient){
  }
}
```

- El proveedor del módulo debe ser declarado en app.config:

```
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [provideHttpClient(),…]
};
```

OBSERVABLES

- Una petición HTTP (get, post, ...) devuelve un Observable
- A un Observable se suscribe el componente controlador para procesar el resultado cuando esté disponible:

componente

```
export class PaisesComponent {
  paises:Pais[];
  constructor(private paisesService:PaisesService){
  }
  cargarPaises():void{
    this.paisesService.obtenerPaises().subscribe(data=>this.paises=data);
  }
}
```

suscripción al observable

servicio

```
export class PaisesService{
  constructor(private http:HttpClient){

  }
  public obtenerPaises(): Observable<Pais[]> {
    return this.http.get<Pais[]>(this.url); //Observable
  }
}
```

Suscripción completa

- **Cuando un cliente se suscribe a un observable, puede indicar que hacer cuando llegue cada dato del flujo, que hacer en caso de error, y que hacer cuando finalice el flujo de datos:**

```
this.paisesService.obtenerPaises().subscribe({
        next: (data) => console.log('Se reciben datos:', data),
        error: (err) => console.error('Se ha producido un error:', err),
        complete: () => console.log('Suscripción finalizada')
});
```

Antonio

Operar con observables

- Antes de suscribirnos a un observable, podemos realizar alguna operación con el observable (filtrar, transformar,..)
- Para ello utilizamos el método *pipe()*, que permite encadenar operadores RxJS sobre un Observable

```
observable.pipe(
  operador1(),
  operador2(),…
)
```

- Cada operador actúa sobre los datos generados por el Observable anterior.
- Este método no modifica el Observable original, cada operador devuelve un nuevo

Antonio Ma

Operadores RxJS

- A continuación, veremos algunos operadores RxJS que se utilizan habitualmente para manejar Observables en Angular:
  - filter(condicion). Devuelve un nuevo Observable con los elementos que cumplen la condición:

```
const numeros$ = of(1, 2, 3, 4, 5);
const resultado$ = numeros$.pipe(
  filter(n => n % 2 === 0),  // filtra los pares
);

resultado$.subscribe(console.log); //muestra 2 4
```

Sierra

Operadores RxJS

- **map(operacion). Devuelve un nuevo Observable resultante de transformar cada elemento del Observable anterior con la operación proporcionada como parámetro:**

```
const numeros$ = of(1, 2, 3, 4, 5);
const resultado$ = numeros$.pipe(
  map(n => n*n),  // cuadrado de cada número
);
resultado$.subscribe(console.log); //muestra 1 4 9 16 25
```

- **take(n). Genera un nuevo observable con los n primeros elementos del observable original**
- **from(iterable). Convierte un iterable o array en un Observable que emite cada elemento del mismo. El operador of() es similar, solo que devuelve todo el iterable de golpe.**

Antonio M

Operadores RxJS

- **reduce(funcion,ini). Acumula cada uno de los valores emitidos por el observable en un elemento auxiliar, aplicando la operación definida por la función. El resultado final es un Observable de un único elemento**

```
const numeros$ = of(1, 2, 3, 4, 5);
const resultado$ = numeros$.pipe(
  reduce((acum,n) => acum+n,0),  // suma de todos los elementos
);
resultado$.subscribe(console.log); //muestra 15
```

```
const cursos = [
      { titulo: 'Angular', precio: 150 },
      { titulo: 'React', precio: 120 },
      { titulo: 'Vue', precio: 100 },
    ];

    from(cursos)
    .pipe(reduce((aux,c)=>aux+c.precio,0))
    .subscribe(r=>console.log(r)); //370
```

Envío de datos en peticiones Http

- **Desde un cliente se pueden enviar datos a un recurso externo de la siguiente manera:**
  - **Path variables. Los datos se envían como parte de la dirección**
  - **QueryString. Se envían parámetros en la URL en parejas clave=valor**
  - **Form-urlencoded. Se envían parámetros en el cuerpo de la petición en parejas clave=valor**
  - **JSON. Se pueden enviar datos como un documento JSON en el cuerpo de la petición**

Antonio Martín Sierra

Path variables y QueryString

- **Path variable:**

```
export class BuscadorService{
    find(cod:number):Observable<Item> {
       return  this.http.get<Item>(`http://localhost:8000/buscador/${cod}`);
    }
}
```

- **Envío como QueryString:**

```
export class BooksService{
    listBooks(isbn: string):Observable<Book[]>{
       return  this.http.get<Book[]>("http://localhost:9000/books",{
           params:{"isbn":isbn}
        });
    }
}
```

FORM-URLENCODED

- **Utilizado habitualmente en peticiones post cuando el backend espera recibir un formulario de datos:**

```
export class EmpleadosService{
    save(cod:number,name:string,age:number):Observable<void>{
       let params=new HttpParams();
       let heads=new HttpHeaders();
       //los parámetros se definen en un objeto params
       params=params.set("codigo",cod);
       params=params.set("nombre",name);
       params=params.set("edad",age);
       //se debe establecer un encabezado con el tipo de contenido
       heads=heads.set("Content-Type","application/x-www-form-urlencoded");
       return this.http.post<void>(url,params,{"headers":heads});
    }
}
```

JSON

- **Forma de envío habitual para enviar un grupo de datos a un servicio REST en el cuerpo de la petición**

```
export class ClientesService{
    save(cliente:Cliente):Observable<void>{
       let heads=new HttpHeaders();
       //se debe establecer un encabezado con el tipo de contenido
       heads=heads.set("Content-Type","application/json");
       return this.http.post<void>(url,cliente,{"headers":heads});
    }
}
```

Cabeceras de respuesta

- **Si queremos tener acceso a las cabeceras de respuesta, se debe incluir el parámetro observe: "response" en la lista de parámetros opcionales de la petición:**

```
buscar(tematica:string):Observable<any>{
    return this.http.get(this.urlBase+"buscar",
         {params:{tematica:tematica},observe:"response"});
  }
```

- **En el componente:**

```
buscar():void{
      this.buscadorService.buscar(this.tematica)
          .subscribe(data=>{
                    console.log(data.headers));  //cabeceras
                    console.log(data.body));   //cuerpo
                    }
   }
```

Routing

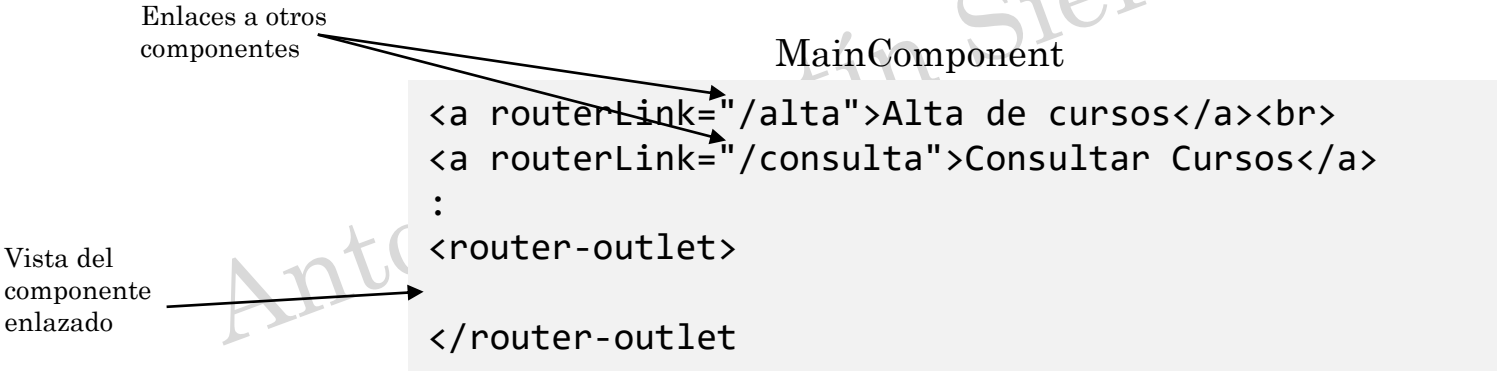
- Es la capacidad para navegar entre componentes
- Enlaces en la vista de un componente provocan la carga de otros componentes
- La navegación puede realizarse también desde código
- El routing es habilitado por defecto al crear la aplicación.
- Las reglas de navegación se definen en el archivo app-routes.ts.

Antonio Martín Sierra



Crear rutas

- Los enlaces a otros componentes se generan en la vista del componente principal mediante el atributo *routelink*.
- En esta vista se incluirá una etiqueta *<router-outlet>* donde se mostrará la vista del componente enlazado:



- La navegación se puede realizar desde código mediante:

```
this.router.navigate(["/eliminar"])
```

Mapeado de rutas a componentes

- **La asociación de rutas a componentes se realiza en el archivo app-routes.ts :**

```
const routes: Routes = [{
  path:"alta",
  component:AltaComponent
},
{
  path:"consulta",
  component: ConsultaComponent
},
{
  path:"eliminar",
  component: EliminarComponent
}

];
```

Configuración

- **En el config.ts se debe registrar el configurador de rutas:**

```
export const appConfig: ApplicationConfig = {
  providers: [ provideRouter(routes),….]
};
```

- **Los componentes que utilicen routing, deben importar el RouterModule:**

```
@Component({
  selector: 'app-home',
  templateUrl: './home.component.html',
  imports: [RouterModule]
})
```

Martín Sierra

Paso de parámetros

- **Se pueden pasar variables por url a través de routerLink:**

```
<a routerLink="['/buscar','Java']">Consultar Cursos</a>
```

- **En la configuración de la ruta:**

```
const routes: Routes = [
  { path: 'buscar/:curso', component: CursoComponent }
];
```

- **En el componente se puede obtener el valor del parámetro:**

```
this.curso = this.route.snapshot.paramMap.get('curso');
```

Antonio Vargas Ibarra

Paso de parámetros

- **También se pueden pasar parámetros por querystring:**

```
<a [routerLink]="['/productos']" [queryParams]="{ categoria: 2 }">
Ver detalles
</a>
```

- **Para obtener los valores en el componente, nos suscribimos a querystring:**

```
this.route.queryParams.subscribe(params => {
  console.log(params['categoria']); // "2"
});
```

Martín Sierra

Almacenamiento de datos

- TypeScript dispone de dos objetos para almacenar datos de usuario como cadenas de caracteres:
  - localStorage. Almacena datos de forma permanente, incluso después de cerrar el navegador
  - sessionStorage. Almacena datos hasta que se cierra la sesión, lo cual ocurre al cerrar el navegador o la pestaña del mismo
- En ambos casos, los datos se almacenan como clave-valor, de tipo string tanto clave como valor

Antonio Martín Sierra

Métodos

- **setItem(key,value). Almacena una cadena con una clave asociada. Los datos se deben almacenar como texto. Si ya existe la clave, se reemplaza el dato:**

```
localStorage.setItem("token","AAFG456RF39340HH");
sessionStorage.setItem("user",JSON.stringify({name="h",role:"promotor"});
```

- **getItem(key). Devuelve un objeto a partir de su clave. Si no existe, devolverá null:**

```
console.log(localStorage.getItem("token"));
const data=JSON.parse(sessionStorage.getItem("user"));
```

- **removeItem(key). Elimina el objeto cuya clave se indica**

tonio Martín Sa

VALIDACIÓN DE FORMULARIOS

- **Angular permite validar formularios sin incluir código :**

Identificadores para acceder a las propiedades de validación

```
<form #form="ngForm" (ngSubmit)="alta(form)">
  Usuario:<input type="text" name="user" #user="ngModel"
                                    [(ngModel)]="user" required><br>
  <div *ngIf="user.invalid && user.touched">
    <small *ngIf="user.errors.['required']">La URL es obligatoria.</small>
  </div>
  :
  <input type="submit" value="Guardar" [disabled]="form.invalid"><br>
</form>
```

Propiedades de validación

El botón se deshabilita si el formulario no es válido

Validación en el componente

- **En ocasiones es necesario tener acceso a las propiedades de validación en el código del componente:**

```
alta(form:any):void{
    if(form.valid){

    }
}
```

Recibe el componente formulario

Se tiene acceso a las mismas propiedades que en la vista

Antonio Martín Sierra

Propiedades de validación

- **Tanto en la vista como en el controlador, se puede utilizar una serie de propiedades del componente:**
  - **invalid. Es true si el componente incumple alguna de las reglas de validación definidas. En el caso del formulario, será true si alguno de los componentes tiene esta propiedad a true.**
  - **valid. Propiedad contraria a la anterior**
  - **touched. True si el componente ha recibido el foco en algún momento**
  - **disabled. Indica si el control está deshabilitado**
  - **errors. Proporciona acceso a todos los errores de validación. Devuelve null si no hay ninguno**

Antonio Martín Sierra

Formularios reactivos

- A diferencia de los estándares, se construyen desde el componente
- Lo controles se comportan como Observables, permitiendo al componente reaccionar a cambios en los mismos.
- Se requiere importar el módulo ReactiveFormsModule en el componente
- Un control se representa como un FormControl y pueden agruparse en FormGroup

Antonio Martín Sierra

Ejemplo formulario reactivo

Componente

Los controles se agrupan en un grupo

Valor inicial del componente

Validadores asignados a cada control

```
export class RegistroComponent{
  registroForm = new FormGroup({
    nombre: new FormControl('', [Validators.required, Validators.minLength(3)]),
    email: new FormControl('', [Validators.required, Validators.email]),
    password: new FormControl('', [Validators.required, Validators.minLength(6)]),
    telefono: new FormControl('', [Validators.required, Validators.pattern('[0-9]{9}')])
  });
  submitted:boolean=false;
  onSubmit() {
    this.submitted = true;
    if (this.registroForm.invalid) {
      console.log('Formulario inválido');
      return;
    }
    console.log('Formulario válido:', this.registroForm.value);
  }
}
```

Ejemplo formulario reactivo

Plantilla

Grupo de controles que formarán el formulario

Nombre del control del grupo al que está asociado

```
<form [formGroup]="registroForm" (ngSubmit)="onSubmit()">
  <label>
    Nombre:<input type="text" formControlName="nombre">
  </label>
  <div *ngIf="registroForm.get('nombre').invalid && submitted">
    <span *ngIf="registroForm.get('nombre').hasError('required')">El nombre es obligatorio.</span>
    <span *ngIf="registroForm.get('nombre').hasError('minlength')">Mínimo 3 caracteres.</span>
  </div>
  <label>
    Email:<input type="email" formControlName="email" [class.email-empresa]="esEmailEmpresa">
  </label>
  <div *ngIf="registroForm.get('email').invalid && submitted">
    <span *ngIf="registroForm.get('email').hasError('required')">El email es obligatorio.</span>
    <span *ngIf="registroForm.get('email').hasError('email')">Formato inválido.</span>
  </div>
   :
  <button type="submit">Registrar</button>
</form>
```

Propiedades/métodos de FormControl

- **Un FormControl tiene una serie de propiedades y métodos para controlar los resultados del formulario reactivo:**
  - **invalid, valid, touched, disabled y errors. Mismo significado que los controles estándares.**
  - **setValidators(). Establece nuevos validadores en el componente.**
  - **clearValidators(). Desactiva los validadores.**
  - **updateValueAndValidity(). Aplica los cambios en las reglas de validación y actualiza el estado.**
  - **valueChanges. Devuelve un observable para suscribirse a los cambios en los valores de las propiedades del componente.**

Antonio Martín Sierra

Signal forms

- Novedad incorporada en Angular 21
- Se basan en la utilización de signals
- El formulario se configura desde el componente mediante funciones de @angular/forms/signals
- Permite también la creación de validadores personalizados mediante la función validate
- Desde la vista se utiliza la directiva [formField] para vincular un control con un elemento de formulario
- Los controles de formulario son tratados como signals

Antonio Martín Sierra

Definición del formulario

controles del formulario

creación del formulario

validadores

validador condicionado

validador personalizado

```
export class App {
    model = signal({
      usuario: '',email: '',profesional: false, instagram: ''
    });

  formClients = form(this.model, (p) => {
    required(p.usuario);
    minLength(p.usuario, 3);
    required(p.email);
    email(p.email);
    required(p.instagram, {
      when: ({ valueOf }) => valueOf(p.profesional)
    });
    validate(p.instagram, ({ value, valueOf }) => {
      if (instagram.includes('@')) {
        return undefined;
      }
      return {
        kind: 'invalido',
        message: 'Debe incluir una @'
      };
    });
  });

}
```

Plantilla del formulario

Formulario estándar

Acceso mediante signals

vinculación a las propiedades del modelo de formulario

```
<div class="container">
    <form (submit)="procesar();$event.preventDefault()" novalidate>
        <label>
          Usuario:<input type="text" [formField]="formClients.usuario">
        </label>
        <br>
        @if(formClients.usuario().invalid()&&error()){
            <span >El nombre de usuario es obligatorio.</span>
        }
        <br>
        <label>
          Email:<input type="text" [formField]="formClients.email">
        </label>
        @if(formClients.email().invalid()&&error()){
            <span >El email es obligatorio.</span>
            <span >Email no válido.</span>
        }
        <br>
        :

        <input type="submit" value="Registrar">
    </form>

</div>
```

Angular Material

- **Biblioteca de componentes UI para crear diseños atractivos y accesibles.**
- **Debe ser instalada con el comando en el raiz de la aplicación o, si esta está en un workspace, en el raiz de este:**

```
>ng add @angular/material
```

- **En el componente donde se vayan a utilizar los objetos material se deberá importar el módulo correspondiente en el decorador:**

```
imports: [
  MatDialogModule
]
```

Antonio Martín Sierra

Cuadro de diálogo

- **Se define como un componente Angular:**

```
export class CuadroDialogoComponent {
constructor(private dialogRef:MatDialogRef<CuadroDialogoComponent>,
    @Inject(MAT_DIALOG_DATA) public data: { mensaje: string }
  ){}
  cerrar() {
    this.dialogRef.close();
  }
}
```

```
<h2 mat-dialog-title>Información</h2>

<mat-dialog-content>
  <p>{{ data.mensaje }}</p>
</mat-dialog-content>
<mat-dialog-actions align="end">
  <button mat-button (click)="cerrar()">Cerrar</button>
</mat-dialog-actions>
```

Martín Cerra

Uso cuadro de diálogo

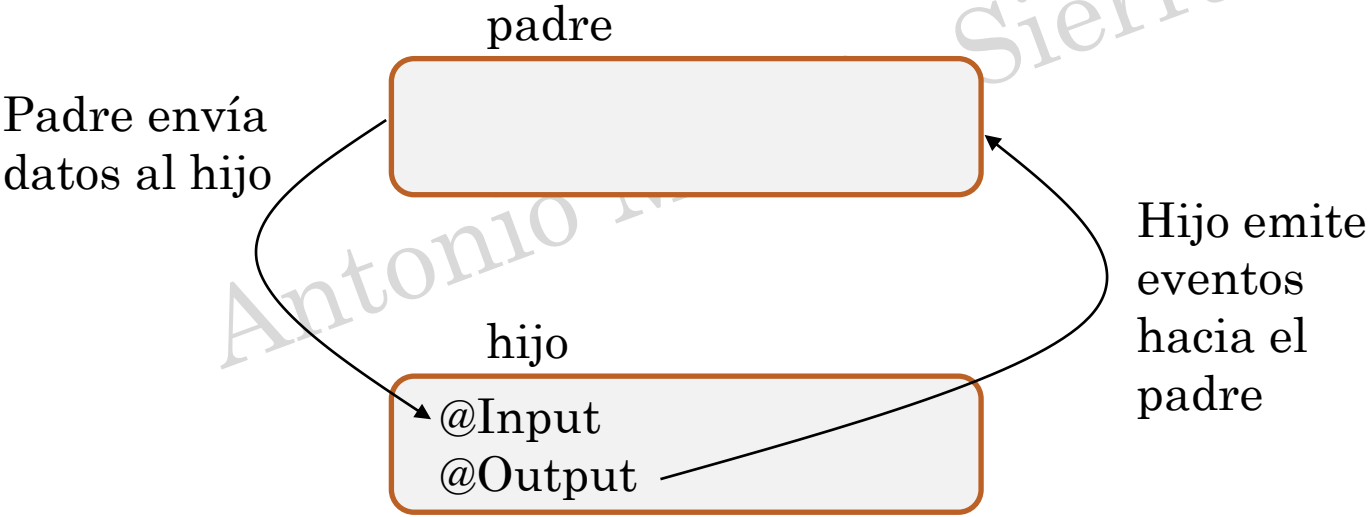
- **Para utilizar el cuadro de diálogo desde otro componente, se debe inyectar MatDialog**

```
constructor(private buscadorService:BuscadorService,private router:Router,
                    private dialog: MatDialog){

  }
  alta(form:any):void{
    if(form.valid){
      this.buscadorService.alta(this.resultado).subscribe({
              next:data=>this.dialog.open(CuadroDialogoComponent,{
                data: { mensaje: "Contacto creado correctamente" }
              }),
              error:err=>this.dialog.open(CuadroDialogoComponent,{
                data: { mensaje: "No se pudo añadir!"+err }
              })
            });
      this.router.navigate(["/buscar"]);
    }else{
      alert("No se pudo enviar el resultado!");
    }
  }
```

Componentes padre-hijo

- **Cuando un componente va a ser reutilizado en distintas partes de la aplicación.**
- **Facilita la modularización de la aplicación**



Antonio M. Sierra

EJEMPLO PADRE-HIJO

componente hijo

```
export class HijoComponent {
  mensaje:string;
  @Input() nombre:string;
  @Output() eventoMensaje:EventEmitter<string>=
            new EventEmitter();
  enviar(){
    this.eventoMensaje.emit(this.mensaje);
  }
}
```

genera el evento
hacia el padre

```
Nombre recibido:{{nombre}}<br><br>
Introduce mensaje para el padre:
<input type="text" [(ngModel)]="mensaje"><br><br>
<input type="button" value="Enviar" (click)="enviar()">
```

componente padre

```
export class AppComponent {
  data:string;
  textoRecibido:string;
  recibir(param:string):void{
    this.textoRecibido=param;
  }
}
```

```
Nombre: <input type="text" [(ngModel)]="data"><br>
<hr>
<app-hijo [nombre]="data"
   (eventoMensaje)="recibir($event)"></app-hijo>
<hr>
Texto recibido del hijo:{{textoRecibido}}
```

Método del padre en el que
se responde al evento
generado por el hijo

variable del padre con
contenido para hijo

Antonio Martín

CARRITO

componente hijo

```
export class ItemProductoComponent {
  @Input() producto: Producto; // Recibe el producto desde el padre
  @Output() agregar = new EventEmitter<Producto>();
  @Output() eliminar = new EventEmitter<number>();
  agregarCarrito() {
    this.agregar.emit(this.producto);
  }
  eliminarCarrito() {
    this.eliminar.emit(this.producto.codigo);
  }
}
```

```
<div class="container">
  <table class="table">
    <tbody>
      <tr>
        <td>{{ producto.producto }}</td>
        <td>{{ producto.categoria }}</td>
        <td>{{ producto.precio }}</td>
        <td><button (click)="agregarCarrito()">Agregar</button></td>
        <td><button (click)="eliminarCarrito()">
          Eliminar</button></td>
      </tr>
    </tbody>
  </table>
</div>
```

componente padre

```
export class CarritoProductosComponent {
  productos: Producto[] = [
    { codigo: 1, producto: "teclado", categoria: "informática", precio: 35 },
    { codigo: 2, producto: "leche", categoria: "alimentación", precio: 1.4 },
    { codigo: 3, producto: "pan", categoria: "alimentación", precio: 2 },
    { codigo: 4, producto: "silla", categoria: "hogar", precio: 110 },
  ];
  carrito: Producto[] = [];
  // recibe el producto desde el hijo y lo agrega
  agregarCarrito(producto: Producto) {
    this.carrito.push(producto);
  }
  // recibe el código del producto y lo borra
  eliminarCarrito(id: number) {
    this.carrito = this.carrito.filter(p => p.codigo != id);
  }
}
```

```
    <tbody>
      <tr *ngFor="let prod of productos">
        <td scope="col" colspan="5">
          <app-item-producto [producto]="prod"
            (agregar)="agregarCarrito($event)"
            (eliminar)="eliminarCarrito($event)">
          </app-item-producto>
        </td>
      </tr>
    </tbody>
:
<table class="table">
    <tr><th>Producto</th><th>Categoria</th><th>Precio</th></tr>
    <tr *ngFor="let item of carrito">
      <td>{{ item.producto }}</td>
      <td>{{ item.categoria }}</td>
      <td>{{ item.precio }}</td>
    </tr>
  <table>
```

Distribuir una aplicación angular

- Se debe traducir todo el código TypeScript+librerías angular a JavaScript, que es lo que entienden los navegadores.
- Para ello se utiliza el comando:

```
>ng build --base-href=/nombre_app/
```

- Se genera un index.html y dos archivos .js en la carpeta dist del proyecto o workspace. Esos archivos se suben a un servidor Web a una carpeta con el nombre_app
- Para probarlo:

```
http://localhost:8080/nombre_app
```

Antonio Martín Sierra

Testing en angular

- La librería jazmin permite realizar pruebas unitarias en Angular
- Se incorpora por defecto con la instalación de Angular cli. Además, utiliza un builder interno para lanzar las pruebas
- Los métodos de test se incluyen en los archivos spec.ts asociados a service y controllers
- Las pruebas se basan en resultados esperados en las llamadas a los métodos

Antonio Martín Sierra

Ejemplo testing de un service

Para ser ejecutado antes de cada test

```
describe('CalculadoraService', () => {
  let service: CalculadoraService;
  beforeEach(() => {
    TestBed.configureTestingModule({});
    service = TestBed.inject(CalculadoraService);
  });
  it("suma de 5 y 6",()=>{
    expect(service.sumar(5,6)).toBe(11);
  })
  it("multiplicacion de 7 y 4",()=>{
    expect(service.multiplicar(7,4)).toBe(28);
  })
  it("factorial de 5",()=>{
    expect(service.factorial(5)).toBe(120);
  })
  it("factorial de -3",()=>{
    expect(service.factorial(-3)).toBe(0);
  })
});
```

- **Para ejecutar las pruebas:**

>ng test
>ng test nombre_app

En aplicaciones dentro de un workspace

Creación de un pipe personalizado

- **Los pipes aplican transformaciones a los datos cuando se van a presentar en la página**

- **Ejemplo:**

nombre del pipe

```
@Pipe({
  name: 'poblacion'
})
export class PoblacionPipe implements PipeTransform {
  transform(value: number): string {
    return value.toLocaleString();
  }
}
```

- **Uso:**

```
<td>{{p.population|poblacion}}</td>
```